

Learning to Unlearn: Machine Unlearning via Learning the Unlearning Behaviors

Hang Zhang
State Key Laboratory for Novel
Software Technology
Nanjing University
Nanjing, China
zhanghang@lamda.nju.edu.cn

Kaifeng Zhang
State Key Laboratory for Novel
Software Technology
Nanjing University
Nanjing, China
zhangkf2022@lamda.nju.edu.cn

Yixiao Ma
State Key Laboratory for Novel
Software Technology
Nanjing University
Nanjing, China
mayx@lamda.nju.edu.cn

Weijie Xu
State Key Laboratory for Novel
Software Technology
Nanjing University
Nanjing, China
xuwj@lamda.nju.edu.cn

Ye Zhu
Centre for Cyber Resilience and Trust
Deakin University
Burwood, VIC, Australia
ye.zhu@ieee.org

Kai Ming Ting[✉]
State Key Laboratory for Novel
Software Technology
Nanjing University
Nanjing, China
tingkm@nju.edu.cn

Abstract

Various machine unlearning techniques have been developed in response to privacy legislation requirements, enabling individuals to exercise their legal right to have their data D_f removed from a machine learning model. This process is typically accomplished via the use of an unlearning function denoted as U . Existing methods focus on designing an intricate U to unlearn $D_f \subset D$ from a previous model $A(D)$, so that the unlearned model performs as closely as possible to the retrained model $A(D \setminus D_f)$. However, these methods often suffer from high computational costs when dealing with massive training data, as the complex structures of U become a bottleneck even for models with fewer parameters. Inspired by Learning to Optimize, we introduce the first learning-based model-agnostic approach, Learning-to-UnLearn (L2UL). Our core insight is to shift from manually designing U to learning the unlearning behaviors from a distribution perspective, thereby acquiring a simple and efficient U via learning. Our experimental results demonstrate that the accuracy achieved by L2UL is comparable to that of retraining while exhibiting impressive efficiency, particularly in data-intensive scenarios. Furthermore, we validate the performance and scalability of our method on larger models ResNet.

CCS Concepts

• Security and privacy; • Computing methodologies → Machine learning; Artificial intelligence;

Keywords

Machine unlearning, Data privacy, Privacy-preserving learning.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Hang Zhang, Kaifeng Zhang, Yixiao Ma, Weijie Xu, Ye Zhu, and Kai Ming Ting. 2018. Learning to Unlearn: Machine Unlearning via Learning the Unlearning Behaviors. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

With the progress of society, there has been a significant increase in the amount of available data. This growth has enriched our lives by allowing us to access a wide range of data shared by numerous individuals online. However, it also presents risks to the privacy of users. In response to this, legislative measures like CCPA (California Consumer Privacy Act), PIPEDA (Personal Information Protection and Electronic Documents Act), and GDPR (General Data Protection Regulation) have been introduced to establish rules that protect users' privacy by the right to delete their data. The task of erasing data from a machine learning model is named machine unlearning [1, 4, 22].

A naive approach is to retrain the model on the remaining data, but this is expensive as models are often trained on very large datasets. Proposing efficient and effective machine unlearning methods has attracted the attention of researchers in recent years.

Current approaches focus on designing a complex unlearning function U so that the unlearned model performs close to the retrained model. For example, some methods design U as a retraining on all or part of the data, which is very time-consuming and requires retraining every time on an unlearning request. Some methods necessitate the calculation of the Hessian matrix for the designed U , which further increases the computational cost. Consequently, even for relatively simple models, the unlearning efficiency remains bottlenecked by the complexity of the designed U , especially when navigating massive datasets.

The goal of an unlearning function U is to produce a model that has unlearned the user data and performs close to the model retrained on the remaining data. **Why don't we learn to unlearn directly?**

Inspired by Learning to Optimize (L2O) [6, 27], traditional optimization algorithms like Adam are manually designed by human

experts through theoretical derivation. In contrast, the L2O approach aims to let machines automatically learn how to optimize. Specifically, the L2O model takes the current state of the optimization iteration as input (such as the current solution and its gradient) and learns to predict the update amount for the next iteration. *Instead of designing a complex and highly time-consuming U , we use the re-trained model as ground truth to learn a U whose structure is simple but enough to unlearn D_f effectively.* In this way, we can achieve the goal of unlearning much more efficiently. Although many methods claim learning to unlearn [5, 17], they usually refer to learning a new decision boundary to achieve forgetting rather than learning the behavior of unlearning.

In this paper, we introduce a model-agnostic framework called Learning-to-UnLearn (L2UL), which employs a machine-learning approach to learn a U .

The main contributions of this work are:

- (1) Pioneering a new framework of machine unlearning via learning, named Learning-to-UnLearn (L2UL).
- (2) Providing the generalization bound of a model unlearned by L2UL in the context of Logistic Regression and Multi-Layer Perceptron (MLP).
- (3) Conducting an empirical validation of the effectiveness and efficiency of our proposed method.

The advantages of the proposed L2UL over current methods are two-fold: (i) Unlike existing methods, in L2UL, the acquisition of U is achieved through learning rather than designing. (ii) Due to the simple structure of U , the execution time of unlearning via the learned U is very short.

2 Problem Formulation

Notations used in this paper are summarized in Table 1.

Table 1: Key notations used in the paper.

Symbols	Definition
$\mathcal{X}$	Sample space
$\mathcal{Y}$	Label space
D	Training set
D_f	Forget set
$A(\cdot)$	Learning algorithm
$U(\cdot)$	Unlearning algorithm
θ	Parameters of the model trained on D
θ'	Parameters of the model trained on $D \setminus D_f$
$\mathcal{P}$	Probability distribution
x	A vector in $\mathbb{R}^d$
κ	Kernel Function
κ_I	Isolation Kernel
t, ψ	Parameter of κ_I

Let $\mathcal{X}$ denote the sample space, and $\mathcal{Y}$ denote the label space. A data point is an element in $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$. A hypothesis function $h : \mathcal{X} \rightarrow \mathcal{Y}$ is learned on training set $D \subset \mathcal{Z}$ by algorithm A to assign $y \in \mathcal{Y}$ to $x \in \mathcal{X}$.

Definition 2.1. A machine learning algorithm is a map A from a subset of $\mathcal{Z}$ to a hypothesis function $h \in \mathcal{H}$, where $\mathcal{H}$ is the space of all hypothesis functions of the algorithm.

After A learned a model from a dataset D , $A(D)$ can be applied to a test set. When some users request to have their data $D_f \subset D$ deleted, model $A(D)$ is obliged to have unlearned their data. The task of unlearning data from a learned model is machine unlearning.

Figure 1 shows the workflow of machine unlearning. To fulfill a request to unlearn D_f from $A(D)$, an unlearning algorithm $U(D, D_f, A(D))$ produces a model which is expected to be the same or close to the model $A(D \setminus D_f)$ retrained on the dataset $D \setminus D_f$.

Definition 2.2. A machine unlearning algorithm is a map $U : 2^{\mathcal{Z}} \times 2^{\mathcal{Z}} \times \mathcal{H} \rightarrow \mathcal{H}$, where $2^{\mathcal{Z}}$ is the power set of $\mathcal{Z}$, which is set containing all of $\mathcal{Z}$'s subset. U takes $D, D_f, A(D)$ as input, and outputs the unlearned model, which is expected to be as close to the retrained model $A(D \setminus D_f)$ as possible.

Definition 2.3. Exact Unlearning: given a randomized learning algorithm A , a dataset D , and a data subset $D_f \subset D$ to be forgotten, an unlearning algorithm U is an exact unlearning process if and only if $\mathcal{P}(U(D, D_f, A(D))) = \mathcal{P}(A(D \setminus D_f))$, where $\mathcal{P}(A(D))$ defines the distribution of all models trained on a dataset D by a learning algorithm A .

The naive unlearning algorithm simply retrains a new model on $D \setminus D_f$ to achieve exact unlearning. However, it is expensive to retrain when $|D \setminus D_f|$ is large.

3 Related Work

Many machine unlearning algorithms that remove some user data from a previously trained model without retraining have been proposed recently. According to the workflow shown in Figure 1, we divide the related work into three categories: D -oriented, D_f -oriented, and $A(D)$ -oriented unlearning.

D -oriented unlearning operates on subsets of D such that only retaining on some subsets is required upon a request to unlearn D_f . For example, SISA [1] divides the training data D into shards and divides each shard into slices. Each shard is used to train a constituent model, incrementally incorporating slices and preserving its parameters until the training set is extended with a new slice. When D_f is required to be forgotten, retraining is limited to the specific constituent model whose shards encompass D_f . Many methods share a similar intuition, such as [2, 4, 10, 13, 15, 38].

D_f -oriented unlearning considers the impact of forgotten data on the model, through influence functions [12, 34, 36] and Markov Chain Monte Carlo-based sampling [9, 21]. When D_f needs to be forgotten, the impact of D_f on the model can be erased accordingly.

$A(D)$ -oriented unlearning considers how the parameters of a learned model should be updated after D_f is forgotten from the training set [18, 20, 25, 28, 37]. [29] updates the parameters by adding noise to the parameters to forget and then tuning the model on the remaining data $D \setminus D_f$. Few-shot learning [8] is used in the case where D is unavailable, and only model $A(D)$ is known.

Although existing works can produce an unlearned model close to the retrained model, none of them use a learning technique to obtain U . Our proposed L2UL, an $A(D)$ -oriented unlearning algorithm,

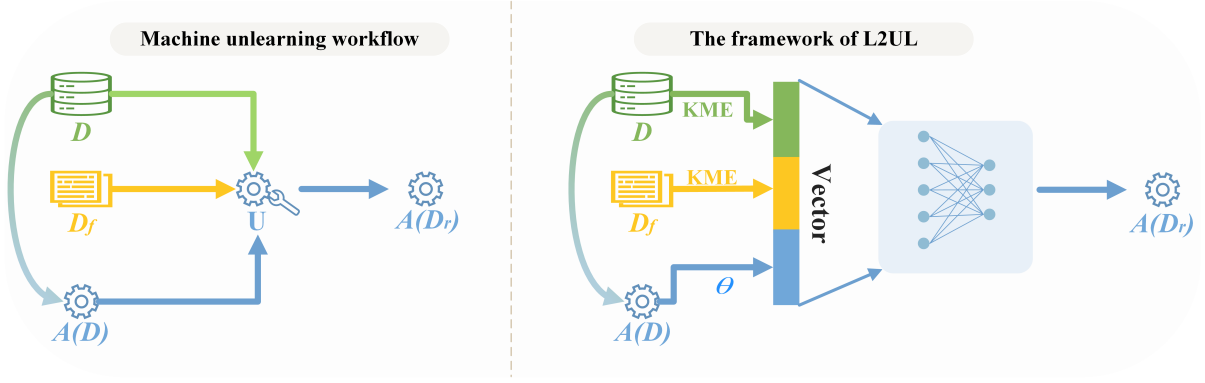


Figure 1: The Machine unlearning workflow and the framework of our proposed method L2UL. The model $A(D)$ is learned on dataset D (which includes D_f) by A . An unlearning algorithm U unlearns D_f from $A(D)$ to obtain a revised model, which is expected to be equal to the retrained model $A(D \setminus D_f)$.

tries to learn the unlearning function U , instead of designing one like current works.

4 Proposed Method

4.1 The Framework of L2UL

Recall Definition 2.2 and Figure 1, U has three inputs, D (represented as a $|D| \times d$ matrix), D_f (represented as a $|D_f| \times d$ matrix) and the parameter vector θ of model $A(D)$. A machine unlearning algorithm U must satisfy the following properties:

Proposition 4.1. *The output of U should remain unchanged after any two rows in the data matrix of D or D_f are exchanged.*

This is easy to understand because exchanging any two rows of D and D_f does not change the dataset.

An effective method that satisfies the Proposition 4.1 is to represent D and D_f as distribution $\mathcal{P}_D$ and $\mathcal{P}_{D_f}$. No matter how the order of points of the dataset changes, its distribution remains the same. Therefore, we redefine Definition 2.2 as:

Definition 4.2. A machine unlearning algorithm is a function $U: \mathcal{P} \times \mathcal{P} \times \mathcal{H} \rightarrow \mathcal{H}$, where $\mathcal{P}$ is the space containing all the probability distributions.

Different from Definition 2.2, Definition 4.2 defines machine unlearning from the perspective of distribution. Based on Definition 4.2 and the unlearning workflow shown in Figure 1, L2UL is naturally proposed by using three components of the unlearning workflow: D , D_f , and $A(D)$. Figure 1 shows the framework of L2UL, which uses a neural network N_U^1 to produce the unlearned model $A_{\theta'}$.

In order to vectorize distribution $\mathcal{P}_D$ and $\mathcal{P}_{D_f}$ as input to N_U , we employ Kernel Mean Embedding (KME) [26] to transform distribution $\mathcal{P}_D$ and $\mathcal{P}_{D_f}$ to vector μ_D and μ_f in Reproducing Kernel Hilbert Space (RKHS).

Kernel mean embedding is a nonparametric method that uses an element of an RKHS as the representation of a probability distribution. The mean feature map $\mu_{\mathbb{P}} = \mathbb{E}[\kappa(\cdot, D)]$ of points X based on

kernel κ embeds distribution $\mathbb{P}$ into RKHS, and preserves all of the statistical features of $\mathbb{P}$.

In our framework, when using some common kernels such as the Gaussian kernel, the Laplacian kernel, etc., $\mu_{\mathbb{P}}$ will map $\mathbb{P}$ into an infinite-dimensional space, which cannot be used as input to the neural network. In other words, we need a kernel whose RKHS dimension is finite. One approach is to use low-rank representations of the kernel matrix. The most popular examples are Nyström method [14, 19, 35] and the Random Fourier Features [24, 39]. However, these are all approximate methods and will reduce effectiveness.

Another approach is to use a kernel with an exact and finite-dimensional feature map. Isolation Kernel [33] is such a kernel that directly obtains the feature map without calculating the kernel matrix, which has been widely used and performs well [3, 31, 40]. Let $D \subset \mathbb{R}^d$ be a dataset sampled from an unknown distribution $\mathbb{P}$. Isolation Kernel uses a partition mechanism such as Voronoi Diagram [23, 41] or hypersphere [32] to partition the data into $|\mathcal{D}|$ cells, where $|\mathcal{D}| = \psi$, and $\mathcal{D} \subset D$ is sampled from D as the seeds of partition mechanism. Let $\mathbb{H}_{\psi}(D)$ denotes the set of all partitions H , each cell $\mathcal{J}[z]$ of partition H isolates $z \in \mathcal{D}$ from the rest of the points in $\mathcal{D}$, the definition of Isolation Kernel is shown as follows:

Definition 4.3. Isolation Kernel: $\forall x, y \in \mathbb{R}^d$, Isolation Kernel of x and y is defined to be the probability that x and y fall into the same cell $\mathcal{J}[z]$ of partition H over all the partitions $H \in \mathbb{H}_{\psi}(D)$:

$$\kappa_I(x, y|D) = \mathbb{E}_{\mathbb{H}_{\psi}(D)} [\mathbf{1}(x, y \in \mathcal{J}[z] | \mathcal{J}[z] \in H)]$$

Given the t partitions $H_1, \dots, H_t$, the feature map $\Phi(x)$ of κ_I is a $\psi \times t$ -dimensional binary column vector.

Definition 4.4. Isolation Kernel's feature map, denoted as $\Phi(x)$, is a vector that represents the cell in which x falls into over partition of t times. For $x \in \mathbb{R}^d$, the dimension of $\Phi(x)$ is $\psi \times t$, where ψ is the number of cells in one partition H . Each element of $\Phi(x)$ is either 0 or 1, indicating the cell in which x falls.

Since the dimension of $\Phi(x)$ is finite, the KME of κ_I can be obtained by simply computing the mean of all feature maps.

Then μ_D, μ_f , and the parameters θ of model A are concatenated as the input $\mathbf{x}$ of the neural network, which contains two hidden

¹Other models can also be used. A two-layer (256, 64) fully connected neural network is used in our experiments.

layers. The KME of distribution $\mathbb{P}$ on the given data D is:

$$\mu_{\mathbb{P}} = \mathbb{E}[\kappa_I(\cdot, D)] = \frac{\sum_{x \in D} \Phi(x)}{|D|}, \quad (1)$$

where $\Phi(X)$ is IK's finite-dimensional feature map determined by two hyperparameters t, ψ . This is also known as the isolation distribution kernel (IDK)[32]. According to Equation 1, we have: $\mu_D = \frac{\sum_{x \in D} \Phi(x)}{|D|}$ and $\mu_f = \frac{\sum_{x \in D_f} \Phi(x)}{|D_f|}$. μ_D and μ_f are later concatenated with the parameters of the original model as input $\mathbf{x}$.

The output θ' is the parameter of the unlearned model $U(D, D_f, A(D))$. And Mean Squared Error (MSE) loss is used as the objective function for training U :

$$\mathcal{L} = \frac{1}{|\theta'|} \sum_{i=1}^{|\theta'|} (\theta'_i - \theta_i^r)^2, \quad (2)$$

where θ^r is the parameters of retrained model $A(D \setminus D_f)$.

Intuitively, we are focusing on models with relatively few parameters but very large training datasets. Because the model has few parameters, U can be learned efficiently, and the large amount of training data can be represented via kernel mean embedding. Therefore, L2UL is effective and highly efficient.

4.2 How to Learn U

In order to train U , first we need a training set $\mathcal{D} = \mathcal{X} \times \mathcal{Y}$. We produce $\mathcal{D}$ by the following three steps. First, we generate $\mathcal{D}$ by randomly sampling s points from D ($s \ll |D|$), and train a model $A(\mathcal{D})$ on $\mathcal{D}$. Second, we randomly select some samples as a forget dataset $\mathcal{D}_f \subset \mathcal{D}$. $\mu_{\mathcal{D}}, \mu_{\mathcal{D}_f}$ of $\mathcal{D}$ and $\mathcal{D}_f$ are obtained using Equation 1. The parameter θ of model $A(\mathcal{D})$ is concatenated with $\mu_{\mathcal{D}}$ and $\mu_{\mathcal{D}_f}$ as $\mathbf{x}$. Finally, we obtain θ^r , the parameter of retrained model $A(\mathcal{D} \setminus \mathcal{D}_f)$. Now we have one instance $(\mathbf{x}, \theta^r) \in \mathcal{X} \times \mathcal{Y}$.

The above steps are repeated m times to obtain $\mathcal{D}$ containing m samples. N_U is later trained on $\mathcal{D}$ by minimizing the loss function $\mathcal{L}$ shown in Equation 2. Since the parameters in KME do not require learning, learning N_U is equivalent to learning U . When there is a D_f that needs to be unlearned from model $A(D)$, we can obtain the unlearned model $A_{\theta'}$ via the learned N_U .

An important point to highlight is that once we have obtained N_U , if we need to unlearn any user data D_f , we can obtain the unlearned model $U(D, D_f, A(D))$ via the learned N_U directly without having to repeat the preprocessing and training phase.

The pseudo code of preprocessing for obtaining $\mathcal{D}$ and $\Phi(\cdot)$ for L2UL, and unlearning D_f are shown in Algorithm 1 and Algorithm 2, respectively.

4.3 Generalization Bound Analysis

Here, we provided the generalization bound of the classifier (Logistic Regression, Multilayer Perceptron) unlearned by L2UL.

Denote the expected loss of Logistic Regression model $A(x|\theta) = \frac{e^{\theta x + b}}{1 + e^{\theta x + b}}$ as: $L(A) = E_{x,y} [y \log(A(x)) + (1 - y) \log(1 - A(x))]$.

L2UL learns the unlearning function U . The expected loss of the model unlearned by U is upper-bounded, as shown in the following Theorem 4.5.

Theorem 4.5.

$$L(U(A(D), D, D_f)) \leq L(A(D \setminus D_f)) + 2R\sqrt{\epsilon},$$

Algorithm 1: Preprocessing

Input : D : Dataset, m : Number of Samples in $\mathcal{D}$, s : size of $\mathcal{D}$, ψ, t : Parameters of κ_I
Output : $\mathcal{D}, \Phi(\cdot)$

- 1 $\mathcal{D} = \{\}$;
- 2 Get the map function $\Phi(\cdot)$ using κ_I ;
- 3 $\forall x \in D$, get the feature map $\Phi(x)$;
- 4 **for** $i \leftarrow 1$ **to** m **do**
- 5 $\mathcal{D} \leftarrow$ randomly sample s points from D ;
- 6 $\mathcal{D}_f \leftarrow$ randomly sample from $\mathcal{D}$;
- 7 $A(\mathcal{D}), A(\mathcal{D} \setminus \mathcal{D}_f) \leftarrow$ train A on $\mathcal{D}, \mathcal{D} \setminus \mathcal{D}_f$;
- 8 $\theta, \theta^r \leftarrow$ Extract parameters of $A(\mathcal{D}), A(\mathcal{D} \setminus \mathcal{D}_f)$;
- 9 $\mu_{\mathcal{D}}, \mu_f \leftarrow$ obtain KME using Equation 1 ;
- 10 $\mathbf{x} \leftarrow$ concatenation of $\mu_{\mathcal{D}}, \mu_f$ and θ ;
- 11 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}, \theta^r)\}$;
- 12 **end**
- 13 Return $\mathcal{D}, \Phi(\cdot)$.

Algorithm 2: Unlearn D_f

Input : $N_U, \mu_D, \Phi(\cdot), A(D)$
Output : $A_{\theta'}$

- 1 $\forall x \in D_f$, get the feature map $\Phi(x)$;
- 2 $\mu_f \leftarrow$ obtain KME of D_f through Equation 1;
- 3 $\mathbf{x} \leftarrow$ concatenation of μ_D, μ_f and $A(D)$'s parameter θ ;
- 4 $\theta' \leftarrow N_U(\mathbf{x})$;
- 5 Return unlearned model $A_{\theta'}$.

where R is the radius of the dataset X ($\forall x \in X, \|x\| \leq R$), and ϵ is the generalization error bound (MSE) of the unlearning model U in L2UL.

Similarly, the expected loss of Multilayer Perceptron (MLP) unlearned by U is also upper-bounded, as shown in Theorem 4.6.

Theorem 4.6.

$$L(U(A(D), D, D_f)) \leq L(A(D \setminus D_f)) + C\sqrt{\epsilon},$$

where L is cross entropy loss, C is a finite scalar determined by the MLP structure and ϵ is the generalization bound (MSE) of the unlearning model U in L2UL.

Both Theorem 4.5 and 4.6 indicate that the expected loss of the classifier unlearned by L2UL is close to that of retrained classifier.

5 Experiments

System: The experiments are executed on a Linux machine with 1T GB RAM and an AMD 128-core CPU, with each core running at 2 GHz.

Data: We use seven public datasets in our experiments². The specifications of the datasets are summarised in Table 2.

²The datasets are available at <https://archive.ics.uci.edu/>, and <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/>

Table 2: Dataset Summary. n=no.instances, d=no.attributes.

	Magic	Adult	Sepsus	Skin	Covetype	SUSY	HIGGS
n	19020	32562	40328	245057	581012	5000000	11000000
d	10	14	89	3	54	18	28

Comparison algorithms: We compare L2UL³ with Retrain, one D -oriented unlearning algorithm: SISA and two $A(D)$ -oriented unlearning algorithms: DeltaGrad, FYEMU.

- (1) **Retrain:** As the most naive approach, it just retrains the model on the remaining dataset $D \setminus D_f$.
- (2) **SISA:** SISA [1] is a well-known framework that partitions the data to reduce the retraining time by just retraining a single model on the shard that needs to be forgotten.
- (3) **DeltaGrad:** DeltaGrad [37] unlearns the data by differentiating the optimization path with the Quasi-Newton method based on information cached during the training phase.
- (4) **FYEMU:** FYEMU [29] is a unlearning algorithm for Neural Networks, which first unlearns data by adding noise to the model parameters, and then obtains a new model through fine-tuning.

Evaluation: We evaluate the efficiency and effectiveness of unlearning algorithms using unlearning time and the accuracy of the unlearned classifier on test data (20%), which is averaged over 10 runs. In each run, we unlearn one randomly selected instance⁴.

5.1 Unlearn linear classifier: Logistic Regression

To begin, we assess the performance of our L2UL approach using a linear classifier known as Logistic Regression (LR). We have demonstrated that the generalization error of the unlearned classifier, denoted as $U(D, D_f, A(D))$, is bounded. Additionally, here we find that L2UL is both efficient and effective when applied to LR.

The results of unlearning one instance in terms of accuracy and unlearning time are shown in Figure 2. L2UL achieves comparable accuracy to Retrain and SISA and outperforms DeltaGrad over all seven datasets. Compared with Retrain, SISA, and DeltaGrad, L2UL speeds up 630, 149, and 282 times on the Magic dataset, and 422500, 49715, and 51500 times on the HIGGS dataset, respectively.

5.2 Unlearn non-linear classifier: Multilayer Perceptron

Nonlinear classifiers can learn complex nonlinear boundaries, which require a more complex process to learn. Besides, a nonlinear classifier usually has more parameters, which makes its parameter space very large. Implementing unlearning in this large space is a more challenging problem.

In this subsection, we test the performance of L2UL on Multilayer Perceptron (MLP) [30]. We use cross-entropy as the loss of the MLP, which has 1 hidden layer with 10 neurons.

The results of unlearning one instance are shown in Figure 3. L2UL achieves comparable accuracy to Retrain and SISA except HIGGS (we will discuss this in Section 5.4). L2UL achieves close

accuracy to DeltaGrad on Sepsis and Skin and outperforms DeltaGrad on five other datasets. L2UL outperforms FYEMU on the two smallest datasets (Magic and Adult) and achieves close accuracy to FYEMU on five other datasets.

Compared with Retrain, SISA, DeltaGrad, and FYEMU, L2UL speeds up 15087, 1203, 730, and 43 times, respectively, on the Magic dataset, and 398000, 105250, 294750, and 32750 times on the HIGGS dataset. The efficiency of L2UL comes from the simple structure of U , which ensures that a single run of U is very quick. Moreover, once U is learned, the parameters of U do not need to change with high probability when a new D_f comes. We will discuss this in Section 8. L2UL just directly uses the learned U to unlearn the new D_f , while the other algorithms require the costly computation again to unlearn D_f . This difference will be more significant when multiple unlearning requests come in sequence.

The results of unlearning 100 and 1000 instances in terms of accuracy and unlearning time are shown in Figure 4 and Figure 5.

5.3 Unlearning Efficacy

In addition to effectiveness and efficiency, the machine unlearning algorithm must also be evaluated to determine whether it has forgotten users' information. We first show the two very commonly used evaluation metrics. Membership Inference Attack [7, 16] (as used in Table 5) and accuracy on unlearned data. The results in terms of Membership Inference Attack and accuracy on D_f ($|D_f|=1000$ for MLP) in Table 3 show that L2UL has forgotten the information of D_f from $A(D)$ ⁵. The U has learned the unlearning behaviors.

However, it is not the case that the lower these two scores are, the better the model performance is, because this may give rise to other issues, such as information exposure [11]⁶. Therefore, we use an example to demonstrate the ability of the proposed algorithm L2UL for calibrating contaminated models, which also shows that L2UL has forgotten D_f .

As previously stated, training data can sometimes be contaminated, which negatively impacts the model's performance. Machine Unlearning is an effective method for cleaning the model when dirty data is detected. A simple example is shown in Figure 6(a). The artificial dataset has two classes, which can be classified by a linear model. But when the training set is contaminated (Figure 6(b) right shows an example with 50 contaminated points), the accuracy of the model will drop.

Figure 6(c) shows the results of LR in terms of Accuracy. When data is not contaminated, the accuracy is 1, as expected. As the number of dirty points grows, the accuracy declines to approximately 0.5. When we use L2UL to clean the contaminated model by treating the dirty points as D_f , the accuracy of the unlearning model remains 1.00 as the number of dirty points grows. Similar results are shown in Figure 6(d) when we replace LR with MLP. The difference is that the accuracy and F1 of MLP decrease more slowly than LR. The accuracy and F1 of the unlearned model are always 1.00. This shows that L2UL has indeed achieved the unlearning of dirty data.

³The codes are available at <https://anonymous.4open.science/r/L2UL>

⁴ $|D_f| = 1$ for LR, $|D_f| = 1$, $|D_f| = 100$, and $|D_f| = 1000$ for MLP.

⁵L2UL classifies all samples into one class on the Sepsis dataset, so MIA cannot be performed

⁶In this paper, if the scores are lower than both the original model ($A(D)$, ORI) and Retrain, it means the unlearning algorithm has forgotten the information.

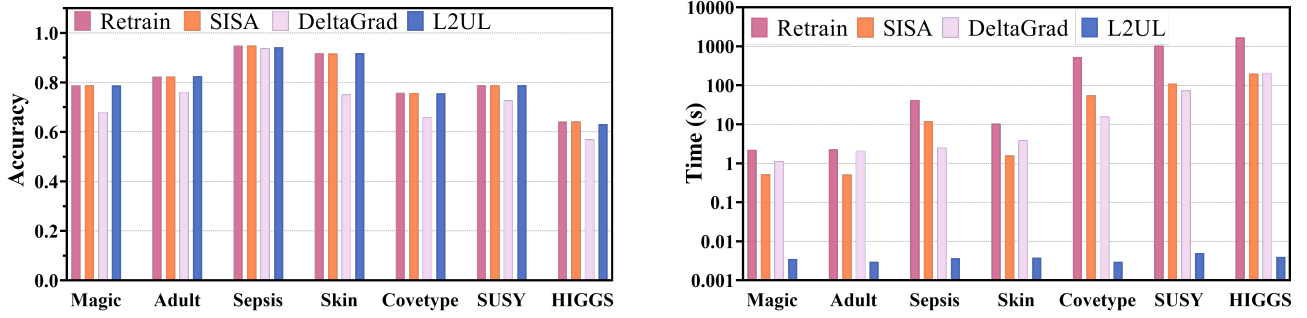


Figure 2: Results of unlearning Logistic Regression.

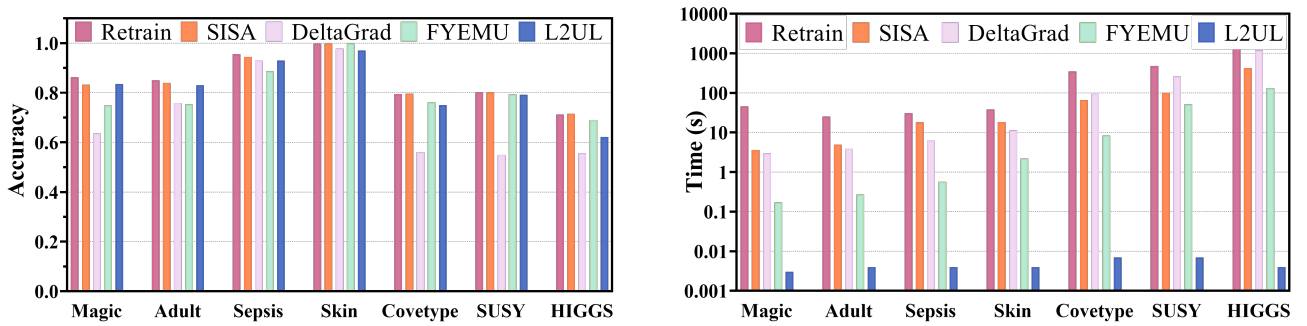
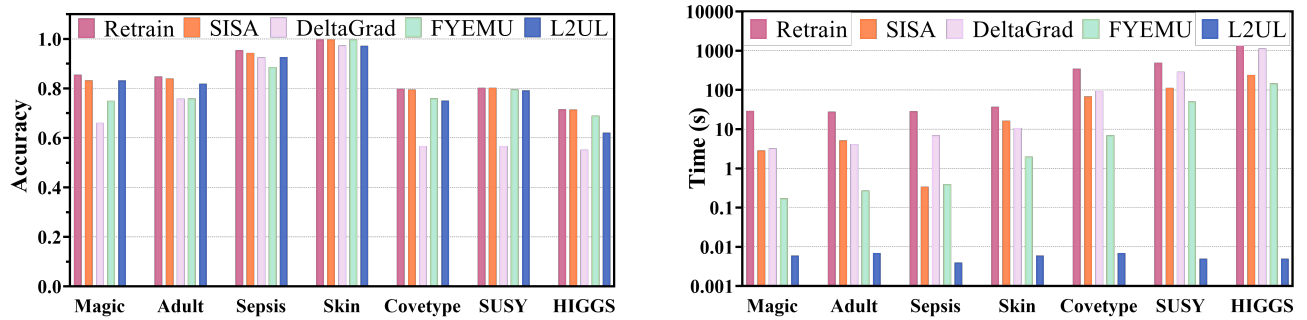
Figure 3: Results of unlearning MLP ($|D_f| = 1$).Figure 4: Results of unlearning MLP ($|D_f| = 100$).

Table 3: Results in terms of Membership Inference Attack and accuracy on unlearn data.

Datasets	MIA			Accuracy on D_f			
	Retrain	FYEMU	L2UL	Retrain	SISA	FYEMU	L2UL
Magic	0.79	0.65	0.75	0.85	0.79	0.71	0.78
Adult	0.80	0.68	0.74	0.81	0.78	0.75	0.76
Skin	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Covtype	0.73	0.68	0.67	0.82	0.76	0.76	0.71
SUSY	0.72	0.69	0.69	0.73	0.71	0.70	0.71
HIGGS	0.65	0.68*	0.50	0.74	0.77*	0.69	0.63

* indicates that the results show that complete forgetting of the data was not achieved.

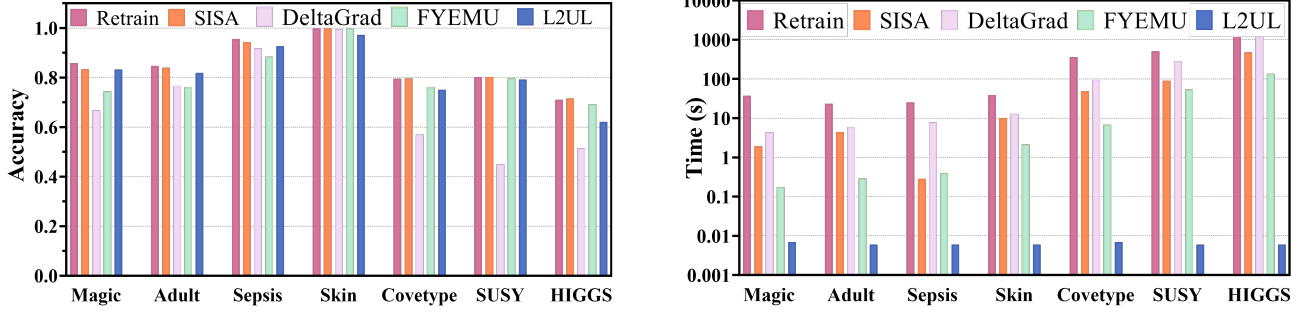
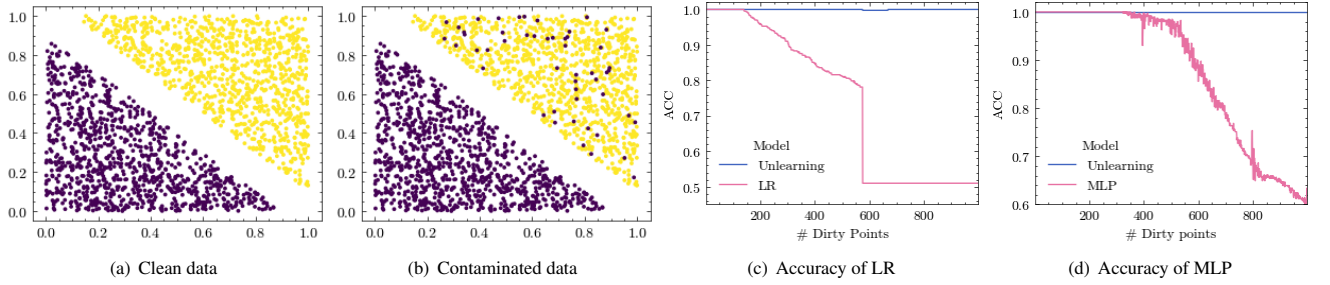
Figure 5: Results of unlearning MLP ($|D_f| = 1000$).

Figure 6: Artificial dataset with clean data and contaminated data, and the results of LR and MLP.

5.4 Parameter Sensitivity Analysis

There are four hyperparameters in the Preprocessing and Unlearning phase of L2UL: ψ , t for Isolation Kernel and s , m for generating $\mathcal{X} \times \mathcal{Y}$. In our experiment, the default setting of hyperparameters is: $\psi = 4$, $t = 100$, $m = 1000$, $s = 1000$ for LR, and $s = 3000$ for MLP.

We report the accuracy of unlearning 1, 100, and 1,000 instances from LR and MLP on the largest datasets SUSY and HIGGS, with $\psi \in [2, 4, 8, 16, 32, 64, 128, 256]$ [32]. [32] shows that the Isolation Distribution Kernel is not sensitive to t , hence, the sensitivity analysis of t is omitted here.

The results of ψ are shown in the first row of Figure 7 and Figure 8, L2UL is robust to ψ when $\psi \geq 4$. The results of s are shown in the second row of Figure 7 and Figure 8. For LR on SUSY, HIGGS, and MLP on SUSY, L2UL is robust to s , and the accuracy of L2UL is close to that of retrain. For MLP on HIGGS, as s increases, the accuracy of L2UL increases from 0.60 to 0.65. This is because when s (sample size) is small, we cannot get a good model $A(\mathcal{D})$ and $A(\mathcal{D} \setminus \mathcal{D}_f)$ as input into L2UL. When sample size s increases, we can have a better model $A(\mathcal{D})$ and $A(\mathcal{D} \setminus \mathcal{D}_f)$, so that a good U and unlearned model $U(D, D_f, A(D))$ with higher accuracy can be obtained on HIGGS.

The results of m are shown in the third row of Figure 7 and Figure 8. L2UL is robust to m , as N_U is a simple model that can achieve good performance with a small dataset $\mathcal{D}$.

In a nutshell, L2UL is robust to the parameters ψ and m . Compared to the parameters of the L2UL model and the parameters of

Table 4: Factor of time complexity (D -or: D -oriented, D_f -or: D_f -oriented, $A(D)$ -or: $A(D)$ -oriented).

U	D -or	D_f -or	$A(D)$ -or	L2UL
Factor	$A \& D$	D_f	$D_f \& \theta$	D_f

IDK, L2UL requires stronger data as input. L2UL learns an unlearning function U , only if the input data $\mathcal{D}$ contains sufficient unlearning information⁷, can a correct and good unlearning function be learned.

6 Complexity Analysis

We show the factor that affects the time complexity of unlearning algorithms as follows (time complexity of the machine learning model A is denoted as $\mathcal{T}(A)$):

- (1) D -oriented unlearning algorithm, such as SISA [1], has a time complexity of $\mathcal{T}(A(|D|/k))$, where k is the number of shards. Because SISA retrains A on some shards, the time complexity is limited by A and $|D|$.
- (2) D_f -oriented unlearning algorithm, such as [12], considers the influence of D_f , whose time complexity is $O(|D_f|d^3)$, which is limited by D_f .
- (3) $A(D)$ -oriented unlearning algorithm, such as [25], considers how to update parameters. Its time complexity is $O(|D_f|d^2 + d^{|\theta|})$ and is limited by D_f and $|\theta|$.

⁷Sufficient unlearning information means a correct and well-trained model $A(\mathcal{D})$ and a correct and well-unlearned model $A(\mathcal{D} \setminus \mathcal{D}_f)$.

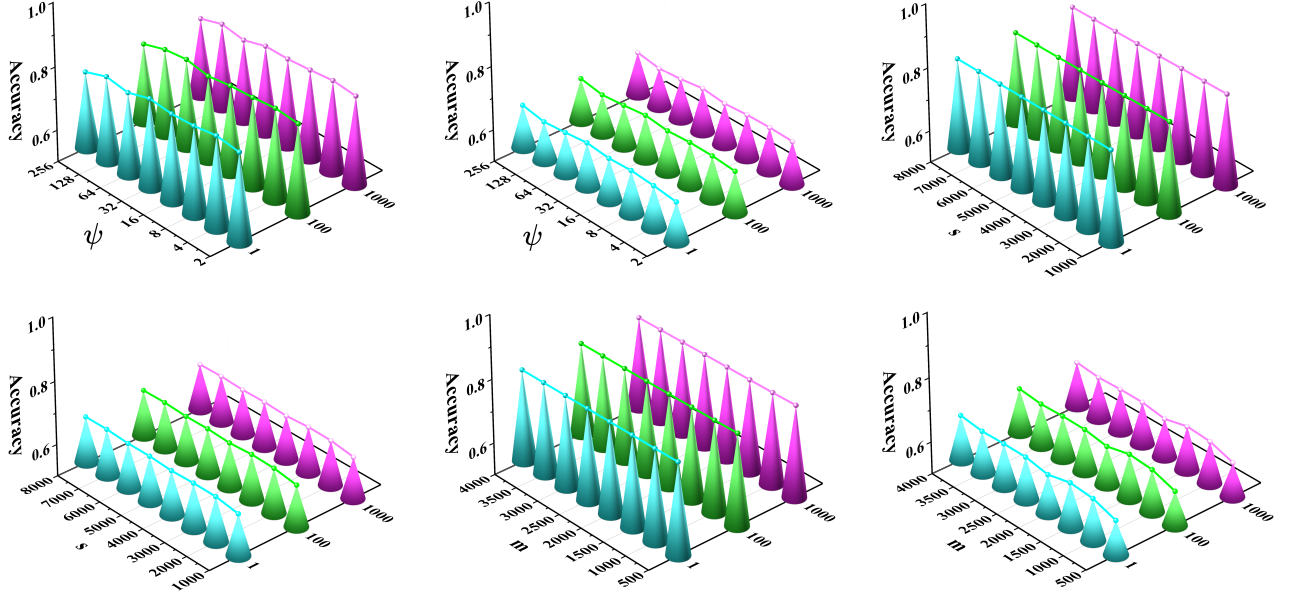


Figure 7: Parameter sensitivity analysis of ψ (top), s (middle), m (bottom) on SUSY (left) and HIGGS (right) for LR.

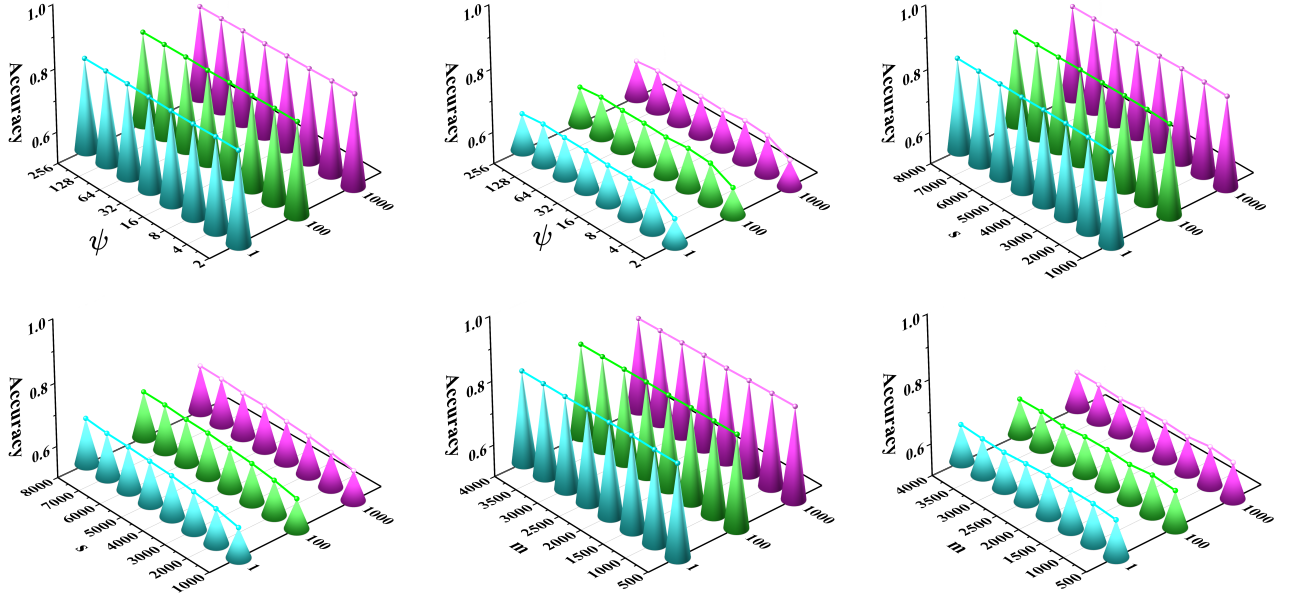


Figure 8: Parameter sensitivity analysis of ψ (top), s (middle), m (bottom) on SUSY (left) and HIGGS (right) for MLP.

The factors of time complexity are summarized in Table 4. Intuitively, D_f is unavoidable because we at least need to know the samples we need to forget. L2UL is also subject to this restriction. Given dataset D , we give the time complexity and space complexity of L2UL as follows:

- * Time complexity of preprocessing is $O(\psi t |D| d + m \mathcal{T}(A(s)))$
- * Time complexity of training U is $\mathcal{T}(U(m))$.

* Time complexity of unlearning D_f from U is $O(\psi t |D_f| d)$.

* Space complexity of U is $O(\psi t d)$.

When unlearning D_f , L2UL needs to calculate the feature map of D_f first. But we can store the feature map of every point from D in advance. In this way, when unlearning D_f , we only need to select the corresponding feature maps and take the mean. Although the time

Table 5: Results of unlearning ResNet-18 on CIFAR-10 dataset.

		10	100	1000	3000
ToW($\uparrow$)	Retrain	1.00	1.00	1.00	1.00
	ADV+IMP	0.32	0.13	0.01	0.01
	RUM	0.60	0.80	0.83	0.83
	L2UL(Ours)	0.80	1.00	0.88	0.99
MIA gap($\downarrow$)	Retrain	0.00	0.00	0.00	0.00
	ADV+IMP	0.30	0.13	0.17	0.14
	RUM	0.30	0.13	0.15	0.12
	L2UL(Ours)	0.30	0.13	0.17	0.14
Time($\downarrow$)	Origin ($A(D)$)	1044.9	-	-	-
	Retrain	1020.5	874.4	783.2	702.7
	ADV+IMP	28.4	78.7	1665.5	12088.2
	RUM	13.6	13.6	13.2	12.8
	L2UL(Ours)	0.5	0.5	0.5	0.6

complexity still depends on D_f , the operation of just taking mean is very fast. As a trade-off, space complexity goes up to $\mathcal{O}(\psi t |D|)$.

7 Unlearn large-scale parameters: ResNet

This paper and the method L2UL we propose primarily focus on the unlearning of simple models (with fewer parameters) trained on large amounts of training data. As demonstrated in the previous sections, the experimental results confirm that L2UL not only maintains high fidelity to the retrained model but also achieves significant gains in computational efficiency. Furthermore, in this section, we verify the effectiveness of our proposed L2UL on a model with a larger parameter size. We tested the performance of the unlearning ResNet-18 model on the CIFAR-10 dataset and compared it with:

ADV+IMP: [5] Misclassify each instance outside of its original prediction, or relabel the instance to a different label. At the same time, use adversarial examples to overcome forgetting at the representation level and use weight importance indicators to accurately locate network parameters that propagate unnecessary information to reduce the time required for forgetting.

RUM: [42] The forgotten set is refined into homogenized subsets based on different features. The meta-algorithm employs the existing algorithms to forget each subset, ultimately providing a model that has forgotten the entire forgotten set.

We train ResNet-18 for 50 epochs on CIFAR-10 with a learning rate of 0.0001. And we report two evaluation metrics, ToW and MIA (Membership Inference Attack) gap, that are used in RUM [42].

The results are shown in Table 5. We highlight the following three key observations: (a) L2UL achieves the highest ToW scores when forgetting 10, 100, 1000, and 3000 samples. (b) L2UL and the comparison algorithms have a closed low MIA gap. (c) L2UL only takes a very short time to complete the unlearning.

In summary, our experimental results demonstrate not only the scalability of L2UL on large-scale data but also its scalability in terms of model parameters. L2UL maintains the performance of the model after forgetting data, and, more importantly, its unlearning efficiency is far superior to existing algorithms.

8 Discussion

8.1 How to learn U when the model cannot be retrained?

It works well for L2UL to use Equation 2 as its loss function only when the parameter of the retrained model $A(\mathcal{D} \setminus \mathcal{D}_f)$ can be obtained. However, the inability to retrain the model is a common challenge faced by machine unlearning. For example, we can't retrain the model on streaming data. In this case, L2UL can no longer be used to generate training data $\mathcal{D}$, and the loss function 2 cannot be used to train N_U . How to find a new loss function when the model cannot be retrained is a challenge.

8.2 Retrain U after unlearning A

It is worth emphasizing that when we employ U to unlearn model, user's information D_f will be included in U . Therefore, we must unlearn U after we use U to unlearn A .

Theorem 8.1. *When sampling m subsets $\mathcal{D}$ from D . The probability that x is sampled at least k_0 times is*

$$\mathcal{P}(k \geq k_0) = \sum_{i=k_0}^m C_m^k \left(\frac{s}{|D|}\right)^i \left(1 - \frac{s}{|D|}\right)^{m-i}.$$

In our experiments, $\mathcal{P}(k \geq 2) < 0.004$ on HIGGS, which means the probability that the data $\{x, y\} \in D$ is used to train U is very low. So we barely need to retrain U . Even if we need, the time cost is affordable, as it only takes 5.2 seconds to retrain U on HIGGS.

8.3 Subdivide the machine unlearning task

Current machine unlearning methods often treat forgetting tasks as a monolithic problem, neglecting the inherent diversity in model complexity and data scale. We argue for a more nuanced, task-specific approach. Since a forgetting task is defined by both the model and the training data, we propose a taxonomy that categorizes machine unlearning into four distinct scenarios based on model size and dataset size:

1) Simple models on small datasets. This represents the most straightforward scenario, where retraining from scratch remains computationally feasible and is often the preferred solution.

2) Complex models on small datasets. Although the training data is limited, the increased model complexity significantly raises the cost of retraining, making it an impractical choice.

3) Simple models on large datasets. This is the focal scenario of our work. Because the model itself is lightweight, applying sophisticated unlearning techniques designed for complex architectures would be unnecessarily resource-intensive—essentially overkill. Instead, a lightweight method, such as the one we propose, is sufficient to achieve effective forgetting.

4) Complex models on large datasets (e.g., large language models). This constitutes the most challenging scenario, demanding advanced and scalable unlearning strategies.

The key advantage of this taxonomy lies in its strategic flexibility: by matching the unlearning method to the specific complexity of the task, we can achieve efficient data removal without incurring prohibitive computational costs, ensuring we neither "use a sledgehammer to crack a nut" nor deploy inadequate solutions for demanding scenarios.

9 Conclusion

In this paper, we propose Learning-to-UnLearn (L2UL), to the best of our knowledge, the first framework to address machine unlearning by learning the unlearning behaviors rather than manually designing complex functions. By prioritizing a distribution-oriented perspective, L2UL achieves a remarkably streamlined yet potent unlearning capability. Focusing on scenarios where the model is simple but the data training dataset is large, we utilized Logistic Regression and Multilayer Perceptrons to demonstrate the ability of L2UL to handle massive datasets with exceptional speed and no loss in accuracy. Both theoretical and empirical results prove that L2UL significantly outperforms existing methods in efficiency. Furthermore, tests on contaminated models and sensitivity analyses confirm the effectiveness and stability of L2UL across various configurations. Finally, successful validation on ResNet underscores the scalability of L2UL to more complex models, offering a robust and efficient solution for modern privacy requirements.

A Proofs

A.1 Proof of Theorem 4.5

Theorem 4.5

$$L(f_{U(A(D), D, D_f)}) \leq L(f_{A(D \setminus D_f)}) + 2R\sqrt{\epsilon},$$

where R is the radius of the dataset X ($\forall x \in X, \|x\| \leq R$), and ϵ is the generalization bound (MSE) of the unlearning model U .

PROOF. Denote $U(A(D), D, D_f) - A(D \setminus D_f)$ as e , then $\|e\| \leq \sqrt{\epsilon}$. By Taylor Expansion, we approximately have

$$\begin{aligned} & L(f_{U(A(D), D, D_f)}) - L(f_{A(D \setminus D_f)}) \\ &= \frac{\partial L(f_{A(D \setminus D_f)})^\top}{\partial A(D \setminus D_f)} e + o(\|e\|), \end{aligned}$$

where $\lim_{\|e\| \rightarrow 0} \frac{o(\|e\|)}{\|e\|} = 0$. Then by Cauchy-Schwarz inequality and convexity of norm function, we have

$$\begin{aligned} & L(f_{U(A(D), D, D_f)}) - L(f_{A(D \setminus D_f)}) \\ &= E_{x,y} [(f_{A(D \setminus D_f)}(x) - y)x^\top] e + o(\|e\|) \\ &\leq \|E_{x,y} [(f_{A(D \setminus D_f)}(x) - y)x]\| \cdot \|e\| + o(\|e\|) \\ &\leq E_{x,y} [\|(f_{A(D \setminus D_f)}(x) - y)x\|] \cdot \|e\| + o(\|e\|) \\ &\leq 2R\sqrt{\epsilon} + o(\sqrt{\epsilon}). \end{aligned}$$

□

Here we refer $U(A(D), D, D_f)$ and $A(D \setminus D_f)$ as the parameters of the unlearned model and retrained model, respectively.

A.2 Proof of Theorem 4.6

Theorem 4.6

$$L(f_{U(A(D), D, D_f)}) \leq L(f_{A(D \setminus D_f)}) + A\sqrt{\epsilon},$$

where A is a finite scalar determined by the MLP structure and ϵ is the generalization bound (MSE) of the unlearning model U .

PROOF. For the $(N+1)$ -layer MLP model (the 0-th layer is the input layer, the $(N+1)$ -th layer is the output layer), we use softmax activation for the output layer, and sigmoid activation function for the rest of the layers. We denote the number of neurons in the i -th

layer as n_i , then the weight between the $(i-1)$ -th layer and i -th layer are W_i of shape (n_i, n_{i-1}) and B_i of shape $(n_i, 1)$. For a finite test set X , $|X| = m$, we denote it as A_0 . Then, the forward propagation scheme is described as follows:

$$Z_k = W_k A_k + B_k, A_k = f_k(Z_k)$$

for $k \in \{1, 2, \dots, N+1\}$, where $f_k(\cdot)$ is the activation function we use in the k -th layer (softmax for the output layer, sigmoid for the rest layers).

For the back propagation, when using Cross-Entropy Loss, we have

$$dZ_{N+1} = A_{N+1} - Y, dW_{N+1} = \frac{1}{m} dZ_{N+1} A_N^\top,$$

$$dB_{N+1} = \frac{1}{m} dZ_{N+1} 1_{(m,1)},$$

and

$$dZ_k = W_{k+1}^\top dZ_{k+1} \odot f'_k(Z_k), \quad (3)$$

$$dW_k = \frac{1}{m} dZ_k A_{k-1}^\top, dB_k = \frac{1}{m} dZ_k 1_{(m,1)}$$

for $k \in \{1, 2, \dots, N, N+1\}$, where d means the derivative of the Cross Entropy Loss, $1_{(m,1)}$ is a column vector of size m , with each entry = 1, and $\odot$ is the element-wise multiplication.

Denote $A(D \setminus D_f)$ as Θ and $U(A(D), D, D_f)$ as $\hat{\Theta}$. We have

$$\Theta = \begin{bmatrix} W \\ B \end{bmatrix}, \quad (4)$$

where W, B are the concatenation of each layer's row-flatten parameter, i.e. $W = [\text{rft}(W_1), \dots, \text{rft}(W_{N+1})]^\top$, $B = [\text{rft}(B_1), \dots, \text{rft}(B_{N+1})]^\top$, where $\text{rft}(\cdot)$ is the operator that flatten a matrix into a row vector. The same notation goes for $\hat{\Theta}$. As in 4.5, we only need to show that $d\Theta$ is bounded, which can be achieved from the bound for dB and dW , since $\|d\Theta\|^2 = \|dW\|^2 + \|dB\|^2$.

For W_{N+1} , we have

$$\|d \text{rft}(W_{N+1})\|^2 = \left\| \frac{(A_{N+1} - Y)A_N^\top}{m} \right\|_F^2 \leq 4n_N n_{N+1}.$$

since the absolute value of each entry in A_{N+1}, A_N, Y is smaller than 1. For the bound on $d \text{rft}W_k$ ($k \neq N+1$), we first need to get the bound R_k for $d \text{rft}Z_k$. We know that

$$\|d \text{rft}(Z_{N+1})\|_\infty \leq R_{N+1} = 2.$$

Then according to Equation 3, we have

$$\|d \text{rft}Z_k\|_\infty \leq \frac{R}{4} n_{k+1} R_{k+1} \triangleq R_k,$$

under the assumption $\|W\|_\infty \leq R$. The iterative form of R_k can be expressed equivalently as

$$R_k = \left(\frac{R}{4}\right)^{N+1-k} \prod_{j=k+1}^{N+1} n_j.$$

Coming back to W_k , for $k \leq N$, we have

$$\|d \text{rft}(W_k)\|^2 \leq R_k^2 n_k n_{k-1}.$$

Finally, we have

$$\begin{aligned} \|d \text{ rft} W\|^2 &= \sum_{k=1}^{N+1} \|d \text{ rft} W_k\|^2 \\ &\leq \underbrace{4n_N n_{N+1} + \sum_{k=1}^N \left(\frac{R^2}{16}\right)^{N+1-k} \frac{\prod_{j=k-1}^{N+1} n_j^2}{n_k n_{k+1}}}_{\triangleq A_W}. \end{aligned}$$

Similarly, we have

$$\|d \text{ rft}(W_k)\|^2 \leq R_k^2 n_k n_{k-1}. \quad (5)$$

Finally, we have

$$\begin{aligned} \|d \text{ rft} B\|^2 &= \sum_{k=1}^{N+1} \|d \text{ rft} B_k\|^2 \\ &\leq \underbrace{4n_{N+1} + \sum_{k=1}^N \left(\frac{R^2}{16}\right)^{N+1-k} \frac{\prod_{j=k}^{N+1} n_j^2}{n_k}}_{\triangleq A_B}. \end{aligned} \quad (6)$$

Then $\|d \Theta\| \leq \sqrt{A_W^2 + A_B^2} \triangleq A$. $\square$

A.3 Proof of Complexity

* Time complexity of preprocessing is $O(\psi t |D| d + m \mathcal{T}(A(s)))$

PROOF. Preprocessing first requires a total time of $O(\psi t |D| d)$ to produce the feature map (line 2 in Algorithm 1), and requires $O(\psi t s + \mathcal{T}(A(s)))$ for each loop. So the time complexity of Algorithm 1 is $O(\psi t |D| d + m \mathcal{T}(A(s)))$. $\square$

* Time complexity of training U is $\mathcal{T}(U(m))$.

PROOF. The time complexity of Train U is using the L2UL to train N_U on the m data points, so the time complexity is $\mathcal{T}(U(m))$. $\square$

* Time complexity of unlearning D_f from U is $O(\psi t |D_f| d)$.

PROOF. Algorithm 2 requires time of $O(\psi t |D_f| d)$ to get the feature map (line 1). and $O(\psi t |D_f|)$ to get μ_f (line 2). Therefore, the time complexity to unlearn D_f from the U is $O(\psi t |D_f| d)$. $\square$

* Space complexity of U is $O(\psi t d)$.

PROOF. we need $O(\psi t)$ to store the μ_D , and $O(\psi t d)$ to store the seeds for partition in isolation kernel. Therefore, the space complexity of U is $O(\psi t d)$. $\square$

A.4 Proof of Theorem 7.1

Theorem 7.1

We sample m subsets $\mathcal{D}$ from D . The probability that sample x is sampled at least k_0 times is:

$$\mathcal{P}(k \geq k_0) = \sum_{i=k_0}^m C_m^k \left(\frac{s}{|D|}\right)^i \left(1 - \frac{s}{|D|}\right)^{m-i}.$$

PROOF. Sampling s points from $|D|$ points, The probability that point x is sampled is $\mathcal{P}(x) = \frac{s}{|D|}$.

Do this sampling m times, the probability that point x is sampled k times is:

$$P(k) = C_m^k \left(\frac{s}{|D|}\right)^k \left(1 - \frac{s}{|D|}\right)^{m-k}.$$

So the probability that sample x is sampled at least k_0 times is:

$$\mathcal{P}(k \geq k_0) = \sum_{i=k_0}^m C_m^k \left(\frac{s}{|D|}\right)^i \left(1 - \frac{s}{|D|}\right)^{m-i}.$$

$\square$

References

- [1] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 141–159.
- [2] Jonathan Brophy and Daniel Lowd. 2021. Machine unlearning for random forests. In *International Conference on Machine Learning*. PMLR, 1092–1104.
- [3] Yang Cao, Haolong Xiang, Hang Zhang, Ye Zhu, and Kai Ming Ting. 2025. Anomaly detection based on isolation mechanisms: A survey. *Machine Intelligence Research* 22, 5 (2025), 849–865.
- [4] Yinzhi Cao and Junfeng Yang. 2015. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*. IEEE, 463–480.
- [5] Sungmin Cha, Sungjun Cho, Dasol Hwang, Honglak Lee, Taesup Moon, and Moontae Lee. 2024. Learning to unlearn: Instance-wise unlearning for pre-trained classifiers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 11186–11194.
- [6] Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. 2022. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research* 23, 189 (2022), 1–59.
- [7] Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanahalli. 2023. Can bad teaching induce forgetting? Unlearning in deep networks using an incompetent teacher. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 7210–7217.
- [8] Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanahalli. 2023. Zero-shot machine unlearning. *IEEE Transactions on Information Forensics and Security* (2023).
- [9] Shaopeng Fu, Fengxiang He, and Dacheng Tao. 2021. Knowledge Removal in Sampling-based Bayesian Inference. In *International Conference on Learning Representations*.
- [10] Antonio Ginat, Melody Guan, Gregory Valiant, and James Y Zou. 2019. Making ai forget you: Data deletion in machine learning. *Advances in neural information processing systems* 32 (2019).
- [11] Aditya Golatkar, Alessandro Achille, and Stefano Soatto. 2020. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9304–9312.
- [12] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. 2020. Certified Data Removal from Machine Learning Models. In *International Conference on Machine Learning*. PMLR, 3832–3842.
- [13] Varun Gupta, Christopher Jung, Seth Neel, Aaron Roth, Saeed Sharifi-Malvajerdi, and Chris Waites. 2021. Adaptive machine unlearning. *Advances in Neural Information Processing Systems* 34 (2021), 16319–16330.
- [14] Sanjiv Kumar, Mehryar Mohri, and Ameet Talwalkar. 2012. Sampling methods for the Nyström method. *The Journal of Machine Learning Research* 13, 1 (2012), 981–1006.
- [15] Huawei Lin, Jun Woo Chung, Yingjie Lao, and Weijie Zhao. 2023. Machine Unlearning in Gradient Boosting Decision Trees. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1374–1383.
- [16] Jiancheng Liu, Parikshit Ram, Yuguang Yao, Gaowen Liu, Yang Liu, PRANAY SHARMA, Sijia Liu, et al. 2024. Model sparsity can simplify machine unlearning. *Advances in Neural Information Processing Systems* 36 (2024).
- [17] Zhuo Ma, Yang Liu, Ximeng Liu, Jian Liu, Jianfeng Ma, and Kui Ren. 2022. Learn to forget: Machine unlearning via neuron masking. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (2022), 3194–3207.
- [18] Zhuo Ma, Yang Liu, Ximeng Liu, Jian Liu, Jianfeng Ma, and Kui Ren. 2022. Learn to forget: Machine unlearning via neuron masking. *IEEE Transactions on Dependable and Secure Computing* (2022).
- [19] Cameron Musco and Christopher Musco. 2017. Recursive sampling for the nystrom method. *Advances in neural information processing systems* 30 (2017).
- [20] Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. 2021. Descent-to-delete: Gradient-based methods for machine unlearning. In *Algorithmic Learning Theory*. PMLR, 931–962.
- [21] Quoc Phong Nguyen, Ryutaro Oikawa, Dinil Mon Divakaran, Mun Choon Chan, and Bryan Kian Hsiang Low. 2022. Markov chain monte carlo-based machine

- unlearning: Unlearning what needs to be forgotten. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. 351–363.
- [22] Thanh Tam Nguyen, Thanh Trung Huynh, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. 2022. A survey of machine unlearning. *arXiv preprint arXiv:2209.02299* (2022).
- [23] Xiaoyu Qin, Kai Ming Ting, Ye Zhu, and Vincent Lee. 2019. Nearest-neighbour-induced isolation similarity and its impact on density-based clustering. In *Proceedings of the 33rd AAAI Conference on AI (AAAI 2019)*, AAAI Press.
- [24] Ali Rahimi and Benjamin Recht. 2007. Random features for large-scale kernel machines. *Advances in neural information processing systems* 20 (2007).
- [25] Ayush Sekhari, Jayadev Acharya, Gautam Kamath, and Ananda Theertha Suresh. 2021. Remember what you want to forget: Algorithms for machine unlearning. *Advances in Neural Information Processing Systems* 34 (2021), 18075–18086.
- [26] Alex Smola, Arthur Gretton, Le Song, and Bernhard Schölkopf. 2007. A Hilbert space embedding for distributions. In *International conference on algorithmic learning theory*. Springer, 13–31.
- [27] Ke Tang and Xin Yao. 2024. Learn to optimize—a brief overview. *National Science Review* 11, 8 (2024), nwae132.
- [28] Ayush Kumar Tarun, Vikram Singh Chundawat, Murari Mandal, and Mohan Kankanhalli. 2023. Deep regression unlearning. In *International Conference on Machine Learning*. PMLR, 33921–33939.
- [29] Ayush K Tarun, Vikram S Chundawat, Murari Mandal, and Mohan Kankanhalli. 2023. Fast yet effective machine unlearning. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [30] Hind Taud and JF Mas. 2018. Multilayer perceptron (MLP). *Geomatic approaches for modeling land change scenarios* (2018), 451–455.
- [31] Kai Ming Ting, Zongyou Liu, Hang Zhang, and Ye Zhu. 2022. A new distributional treatment for time series and an anomaly detection investigation. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2321–2333.
- [32] Kai Ming Ting, Bi-Cun Xu, Takashi Washio, and Zhi-Hua Zhou. 2020. Isolation Distributional Kernel: A New Tool for Kernel based Anomaly Detection. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 198–206.
- [33] Kai Ming Ting, Yue Zhu, and Zhi-Hua Zhou. 2018. Isolation kernel and its effect on SVM. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2329–2337.
- [34] Alexander Warnecke, Lukas Pirch, Christian Wressnegger, and Konrad Rieck. 2023. Machine unlearning of features and labels. *Network and Distributed System Security (NDSS) Symposium* (2023).
- [35] Christopher Williams and Matthias Seeger. 2000. Using the Nyström method to speed up kernel machines. *Advances in neural information processing systems* 13 (2000).
- [36] Ga Wu, Masoud Hashemi, and Christopher Srinivasa. 2022. Puma: Performance unchanged model augmentation for training data removal. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 8675–8682.
- [37] Yinjun Wu, Edgar Dobriban, and Susan Davidson. 2020. Deltagrad: Rapid re-training of machine learning models. In *International Conference on Machine Learning*. PMLR, 10355–10366.
- [38] Zhaomin Wu, Junhui Zhu, Qinbin Li, and Bingsheng He. 2023. DeltaBoost: Gradient Boosting Decision Trees with Efficient Machine Unlearning. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.
- [39] Tianbao Yang, Yu-Feng Li, Mehrdad Mahdavi, Rong Jin, and Zhi-Hua Zhou. 2012. Nyström method vs random fourier features: A theoretical and empirical comparison. *Advances in neural information processing systems* 25 (2012).
- [40] Hang Zhang, Kai Ming Ting, and Ye Zhu. 2025. Kernel-bounded clustering: Achieving the objective of spectral clustering without eigendecomposition. *Artificial Intelligence* (2025), 104440.
- [41] Hang Zhang, Kaifeng Zhang, Kai Ming Ting, and Ye Zhu. 2023. Towards a Persistence Diagram that is Robust to Noise and Varied Densities. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 41952–41972.
- [42] Kairan Zhao, Meghdad Kurmanji, George-Octavian Bărbulescu, Eleni Triantafillou, and Peter Triantafillou. 2024. What makes unlearning hard and what to do about it. *Advances in Neural Information Processing Systems* 37 (2024), 12293–12333.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009